

Task A

Part 1_ Load and prepare my dataset

```

15 # Begin your code (Part 1)
16 dataset = list()
17 for pic_name in os.listdir(r"./" + data_path + r"/car/"):          #use r to un-escaped
18     img = cv2.imread(r"./" + data_path + r"/car/" + pic_name, cv2.IMREAD_GRAYSCALE)
19     img = cv2.resize(img, (36, 16), interpolation = cv2.INTER_AREA)
20     dataset.append((img,1))
21
22 for pic_name in os.listdir(r"./" + data_path + r"/non-car/"):      #use r to un-escaped
23     img = cv2.imread(r"./" + data_path + r"/non-car/" + pic_name, cv2.IMREAD_GRAYSCALE)
24     img = cv2.resize(img, (36, 16), interpolation = cv2.INTER_AREA)
25     dataset.append((img,0))
26

```

建立空 list 後，先用 os.listdir 讀取路徑後，再以迴圈方式讀取路徑下所有檔案。以灰階讀入後再以 INTER_AREA 方式縮放至 36*16 的大小，並加入 dataset 中。Label 部分讀取 car 訓練資料時為 1，non-car 訓練資料時則設為 0

Part 2_Build and Train Models

```

20 # Begin your code (Part 2-1)
21 trainImgArr = np.array(list(zip(*train_data))[0])
22 trainImgInx = np.array(list(zip(*train_data))[1])
23 trainShape = (len(train_data), 16 * 36)
24 testImgArr = np.array(list(zip(*test_data))[0])
25 testImgInx = np.array(list(zip(*test_data))[1])
26 testShape = (len(test_data), 16 * 36)
27
28 self.x_train = np.reshape(trainImgArr, trainShape)
29 self.y_train = trainImgInx
30 self.x_test = np.reshape(testImgArr, testShape)
31 self.y_test = testImgInx
32
33 # raise NotImplementedError("To be implemented")
34 # End your code (Part 2-1)
35
36 self.model = self.build_model(model_name)

```

將 dataset 中 tuples，利用 zip*方法轉置陣列以 numpy array 儲存，再以 reshape 方法將訓練照片、測試照片整理為二維陣列；訓練標籤、測試標籤整理為一維陣列，以符合後續 scikit-learn 模組訓練時需要引入的參數，注意總像素數必須維持不變，否則會出錯。

```
39 def build_model(self, model_name):
40     '''
41     According to the 'model_name', you have to build and return the
42     correct model.
43     '''
44     # Begin your code (Part 2-2)
45     if(model_name == "KNN"):
46         model = KNeighborsClassifier(n_neighbors = 1, p=1)
47     elif(model_name == "RF"):
48         model = RandomForestClassifier(n_estimators = 100,criterion='entropy',)
49     elif(model_name == "AB"):
50         model = AdaBoostClassifier(n_estimators=60)
51     elif(model_name == "GB"):
52         model = GradientBoostingClassifier(n_estimators=140)
53     else:
54         raise ValueError("Invalid model names: {model_name}")
55     return model
56     raise NotImplementedError("To be implemented")
57     # End your code (Part 2-2)
```

根據傳入的模組名稱，將 model 指定為該分類器並調整相關參數以達到最佳預測效果。

```
def train(self):  
    '''  
    Fit the model on training data (self.x_train and self.y_train).  
    '''  
    # Begin your code (Part 2-3)  
    self.model.fit(self.x_train, self.y_train)  
    # raise NotImplementedError("To be implemented")  
    # End your code (Part 2-3)
```

在指定模組與訓練超參數後，使用 scikit-learn 的 fit 方法進行訓練。

問題回答：

1. 參數模型預設數據符合已知的某種機率分布，然後在訓練過程中找出符合這些分布的參數。優點為簡潔、快速、數據需求量較少；缺點則為能處理的複雜度有限、若資料不符合分布易出現過擬合或欠擬合的狀況。舉例：以房屋大小與售價的線性關係預估售價。

非參數模型則相反，不對資料分布作任何或過多假設，而是根據訓練資料擬合學習出某種函數。優點為可用範圍大、對訓練樣本擬合性強；缺點則為數據需求量大、訓練過程長、易過擬合等。舉例：使用 KNN 搭配交叉驗證訓練模型。

2. 集成學習透過結合許多監督式學習的模型來達到更好的預測效果，分為自助聚合(bagging)、提升法 (boosting)與堆疊法 (stacking)三種，bagging 透過不斷將訓練資料重新採樣，產生不同組的訓練資料，再對每組資料產生模型，透過平均或多數決的方式產生最終的結果。Ex:RF；boosting 則依據每筆訓練資料的難易度分配權重，針對較難的資料(預測錯的)分配加強權重重新訓練，來達到越來越準確的目標。Ex:Adaboost。Stacking 的每個模型間沒有關聯，把不同種模型的預測結果當成新的模型輸入，來建立最終預測結果的模型。

3. n_neighbor 表示 KNN 分類器中，判斷當前資料時參考的周遭資料點數量，根據他們的 label 來進行預測；n_estimator 在 RF 中代表分類器要建立多少決策樹，即要重新採樣的組數、最後跟據多少組資料的平均或多數決以產生最後的預測結果；在 Adaboost 中則表示弱分類器的數量，像上面提到的跟據弱分類器預測的結果提供反饋，不斷調整權重產生強分類器，產生最後的預測結果。

4. Confusion Matrix 將預測結果分為四類：

True Positive(TP)、False Negative(FN)、False Positive(FP)、True Negative(TN)，用來表達預測結果與真實結果間的交互關

係，TP 表皆為是，FN 表預測為否但真實為是，FP 表預測為是但真實為否，TN 表皆為否。以這次的 Task 輸出來說，左上表 TP、右上表 FN，左下表 FP，右下表 TF，T、P 表車位上有車，F、N 表車位沒車。

5. Accuracy 公式： $TP+TN/TP+FP+FN+TN$

Precision 公式： $TP/TP + FP$

Recall Rate 公式： $TP/TP + FN$

由公式可知，Accuracy 追求所有情況下，整體展現的準確度；

Precision 追求預測是真的時候，有多少符合真實的真

Recall Rate 追求真實的真發生時，預測是真的比例

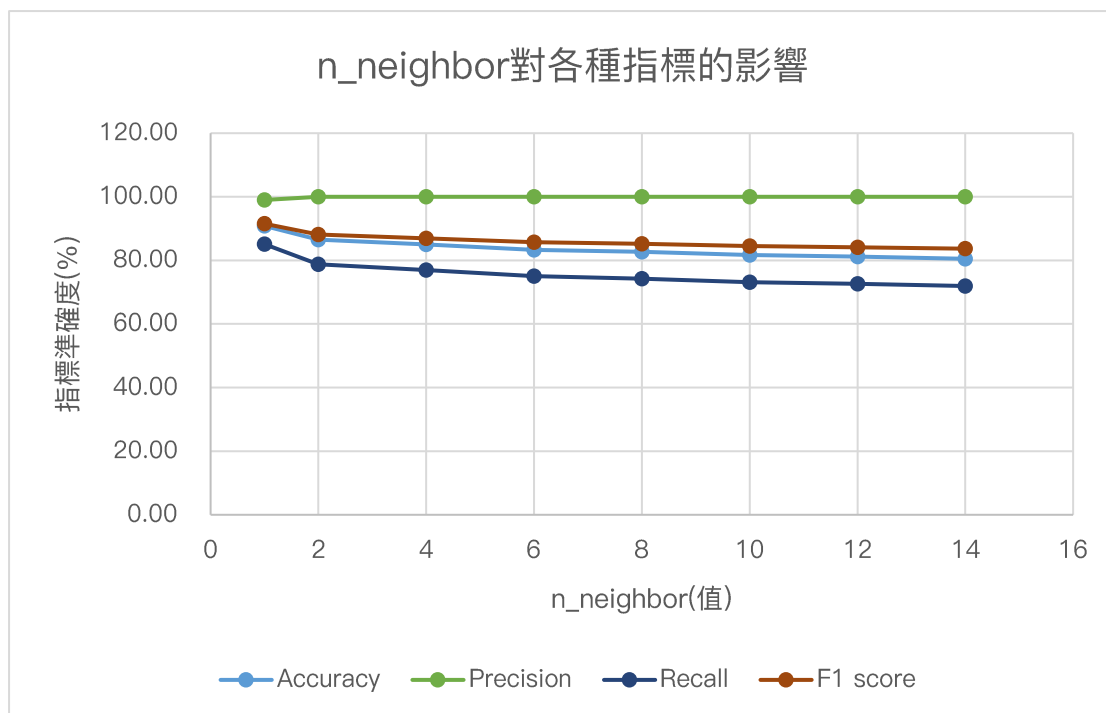
Precision 高表達 False alarm 低

recall rate 高表達 miss alarm 低

舉例：找尋可能毀滅地球的隕石，寧可錯找 10 顆不能漏掉 1

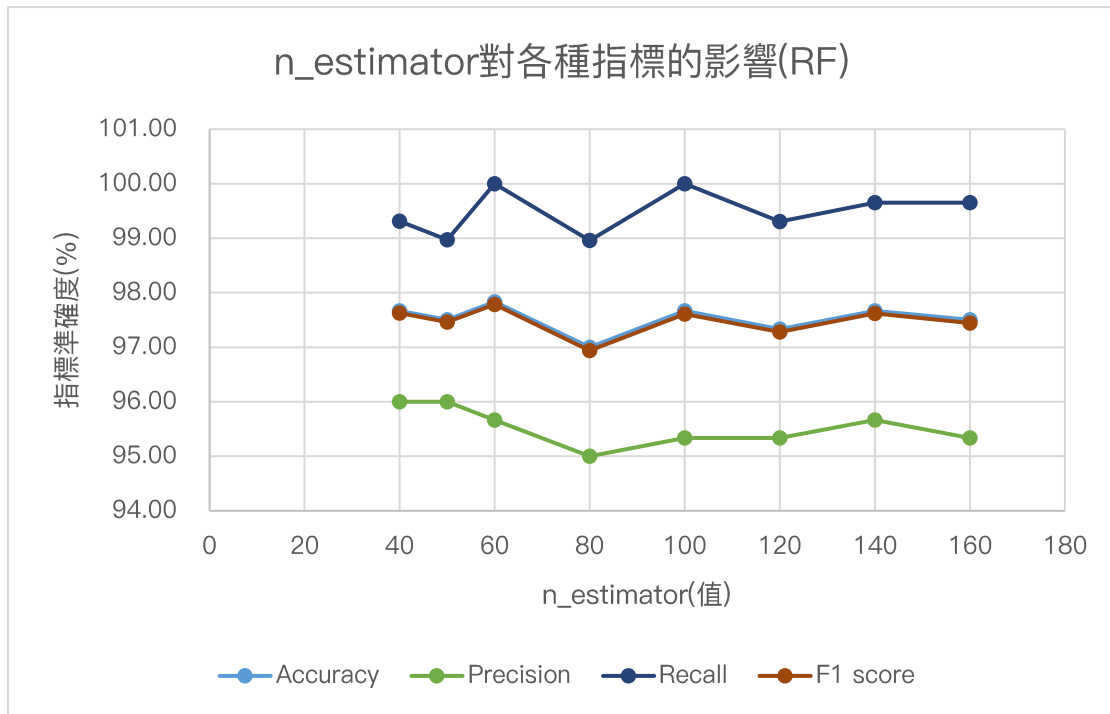
顆，追求 Recall Rate 優先；預測選定天氣好的日子出遊，可能好天氣不出遊是可以被接受的，但出遊卻壞天氣糟透了，此時追求 Precision 優先。

Part 3_Additional Experiments



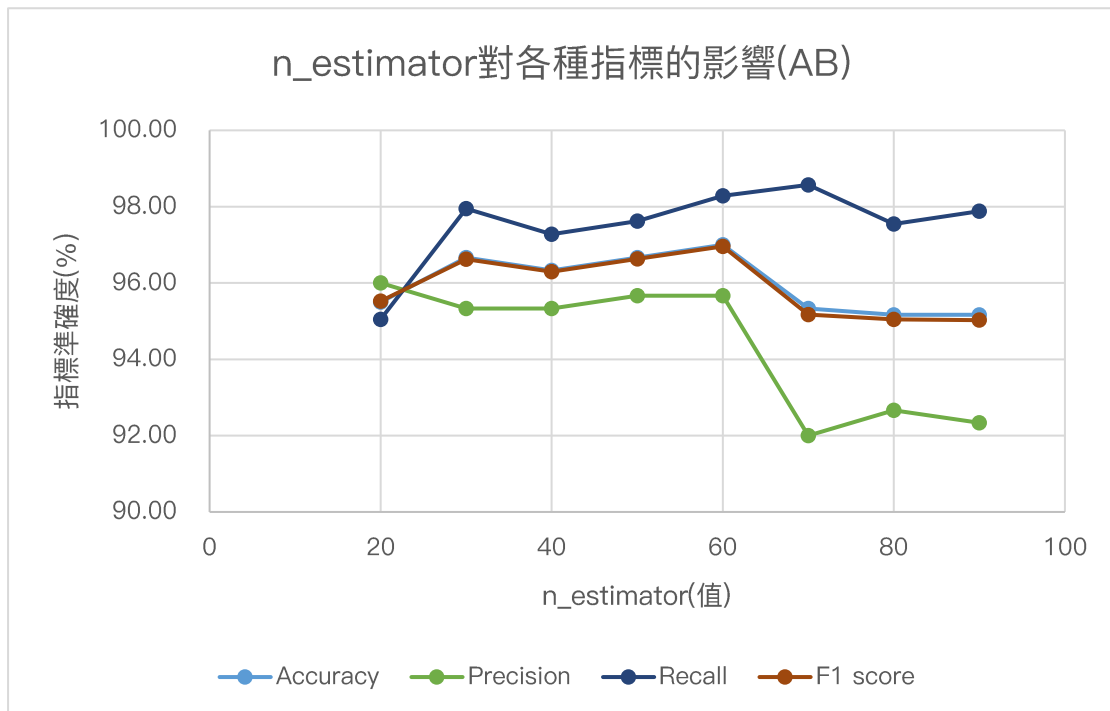
圖一：n_neighbor 值對各種指標的影響

由上圖可見 n_neighbor 越大，Accuracy, Recall 與 F1_score 都逐漸下滑；Precision 則無明顯變化(略增後至頂)。此例子中 p 值皆 = 1，因測試過 p = 1 較 p = 2、3 整體預測較準確。



圖二：n_estimator 值對各種指標的影響(RF)

由上圖可見 n_neighbor 越大，Accuracy, Recall 與 F1_score 有震盪情形後下降；Precision 則逐漸降低。



圖三：n_estimator 值對各種指標的影響(AB)

由上圖可見 $n_neighbor$ 越大，四者皆先迎來上升；除 Recall 值在 70 達到高峰後下滑，其餘在 60 後接下滑。

最後選擇 RF $n_estimator = 100$, $criterion = entropy$ 來 detect

Part 4_Detect Car

```
50 # Begin your code (Part 4)
51 fileIn = open('./data/detect/detectData.txt', 'r', encoding='utf-8')
52 fileOut = open('./data/detect/ML_Models_pred.txt', 'w', encoding='utf-8')
53 detected = fileIn.read()
54 cord = re.split(r'[\s]', detected)
55 cord.pop(len(cord)-1)
56 cord = list(map(int, cord))
57 toatalSpace = cord.pop(0)
58
59 cap = cv2.VideoCapture(r'./data/detect/video.gif')
60 while True:
61     ret, frame = cap.read()
62     if ret:
63         for count in range(toatalSpace):
64             imgCrop = crop(cord[count * 8], cord[count * 8 + 1], cord[count * 8 + 2], cord[count * 8 + 3], cord[count * 8 + 4],
65                             cord[count * 8 + 5], cord[count * 8 + 6], cord[count * 8 + 7], frame)
66             imgCrop = cv2.cvtColor(imgCrop, cv2.COLOR_BGR2GRAY)
67             imgCrop = cv2.resize(imgCrop, (36, 16), interpolation=cv2.INTER_AREA)
68             imgCrop = imgCrop.reshape(1, 36 * 16)
69             if clf.classify(imgCrop):
70                 spaceOccuCord = np.array([cord[count * 8], cord[count * 8 + 1], cord[count * 8 + 2], cord[count * 8 + 3], cord[count * 8 + 6],
71                                             cord[count * 8 + 7], cord[count * 8 + 4], cord[count * 8 + 5]])
72                 spaceOccuCord = spaceOccuCord.reshape((-1, 1, 2))
73                 cv2.polylines(frame, [spaceOccuCord], True, (0, 255, 0), 2)
74
75                 fileOut.write(str(clf.classify(imgCrop)))
76                 if count != toatalSpace-1:
77                     fileOut.write(" ")
78                 fileOut.write("\n")
79                 cv2.imshow('detect', frame)
80                 key = cv2.waitKey(500)
81                 if key == 27:
82                     break
83             else:
84                 break
85
86 # raise NotImplementedError("To be implemented")
87 # End your code (Part 4)
```

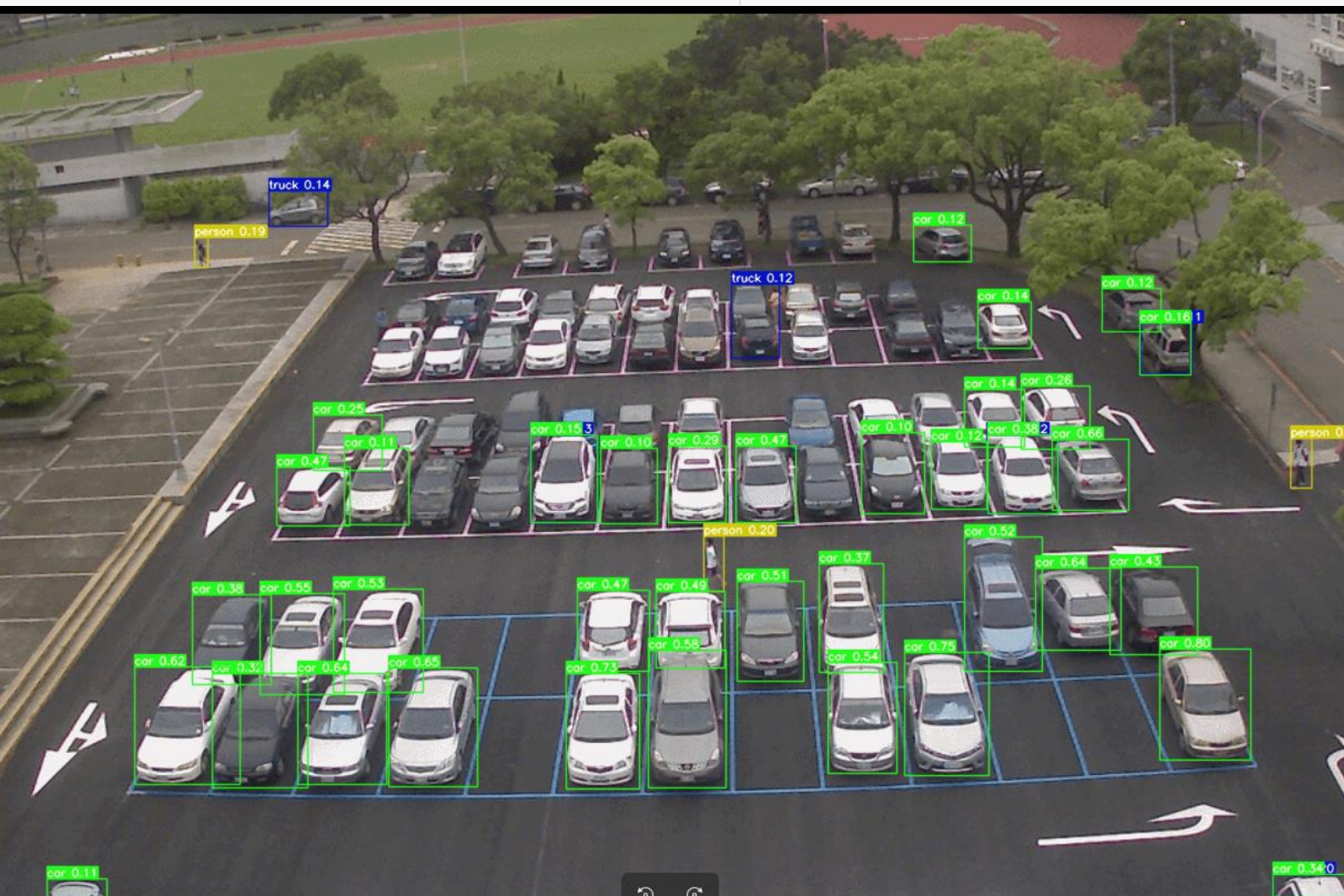
將影片讀入後，利用 `re.split` (Python build in)方法將座標依空格分割存入列表，首項為停車格數，存好後即可移出。接著運用切割方法將每個停車格取出後，縮放並轉成灰階以進行判斷，如果判斷為有佔用中，則以 `polylines` 方法標註綠框，並將 `lable` 值寫出至 `ML_model_pred` 檔，同步預覽影片以確認效果是否正確。

Task B

Part 1_Load a pretrained model and directly apply it to the .png
to detect the object.

Code Implement:

```
'''  
unzip the file HW1_material.zip which placed in your google drive ex: /content/gdrive/MyDrive/hw1/HW1_material.zip to /content/YoloV7  
'''  
!unzip '/content/gdrive/MyDrive/hw1/HW1_material.zip' -d'/content/yolov7'
```



圖四：Yolov7 產出圖片

Accuracy that above 90%:

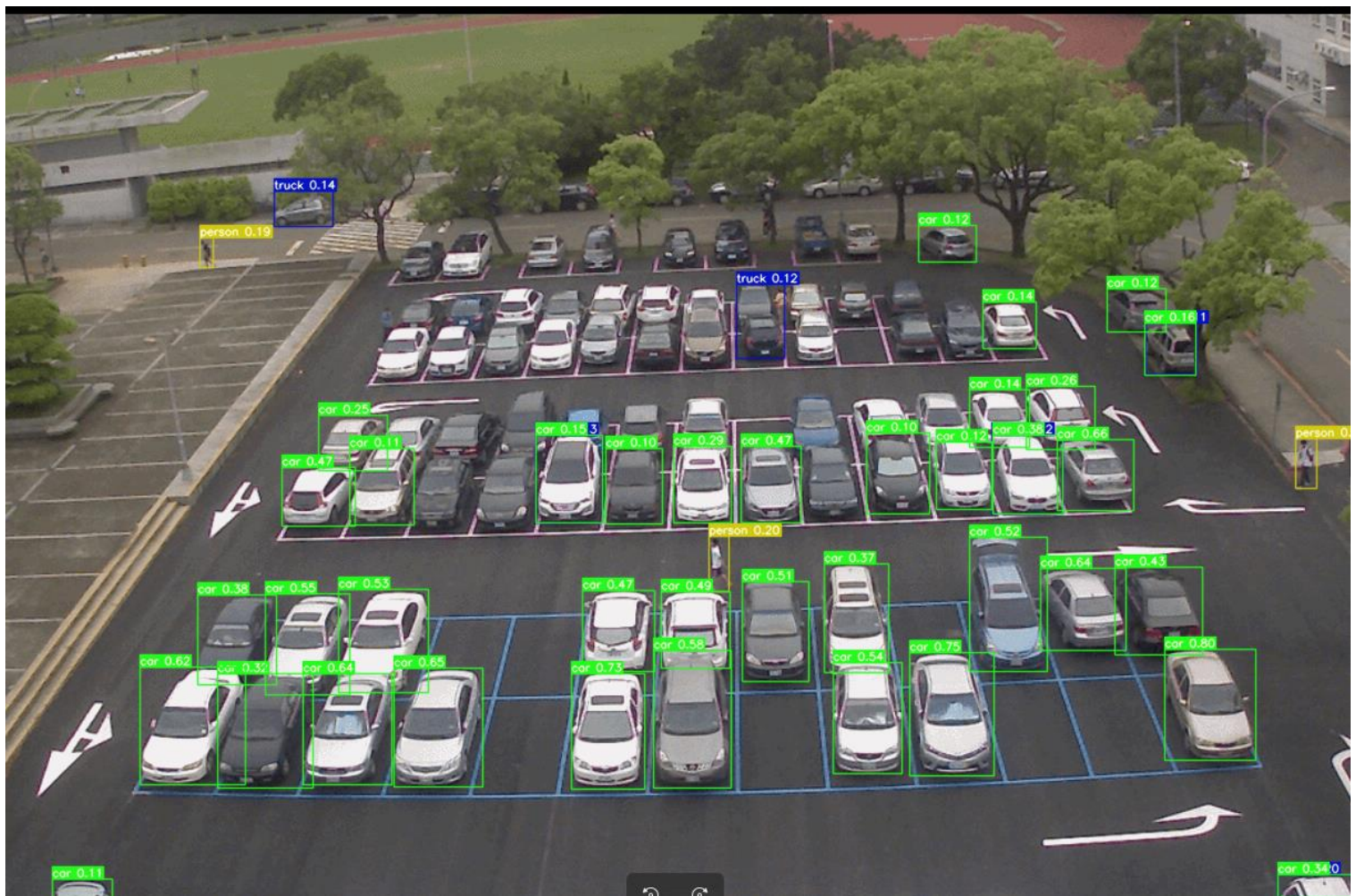
```
Calculate('/content/yolov7/HW1_material/test/', '/content/yolov7/runs/detect/test/labels/')
```

False Positive Rate: 0/300 (0.000000)

False Negative Rate: 2/300 (0.006667)

Training Accuracy: 598/600 (0.996667)

With epoch = 300, 90% can be easily reached. Such a powerful model.

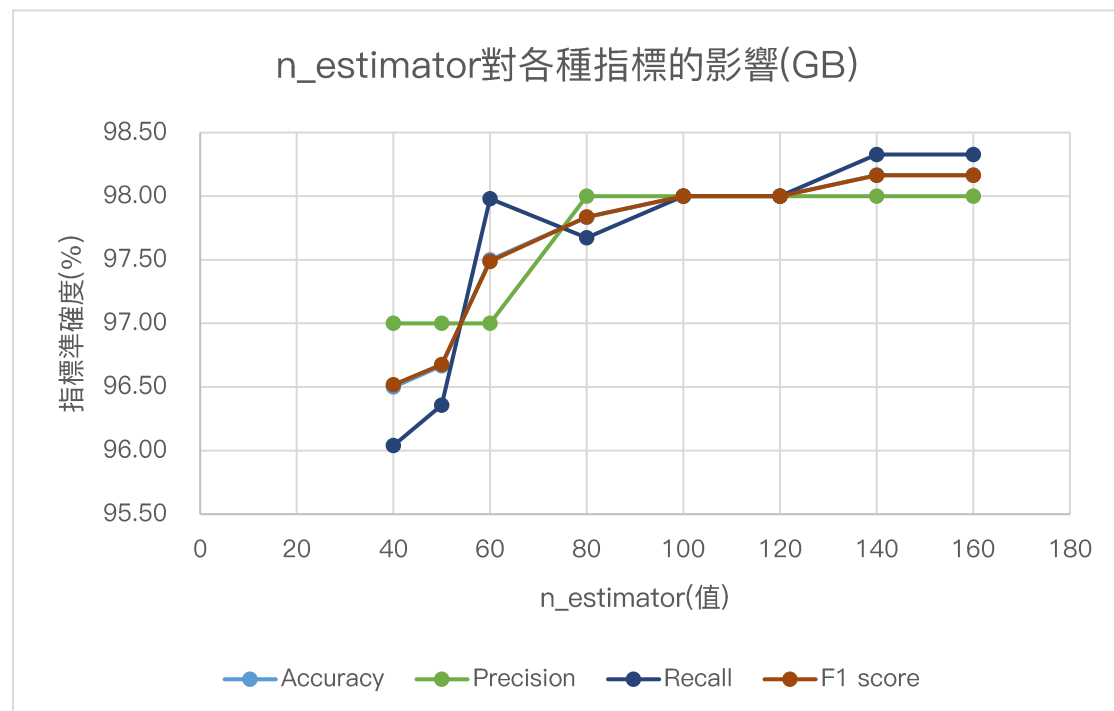


圖五：fine tuning 後 YOLOv7 產出圖片

可以看到還有很多的车位有車但沒有被判斷出來，有車的閾值應該可以再下降一點，會更貼近真實狀況。

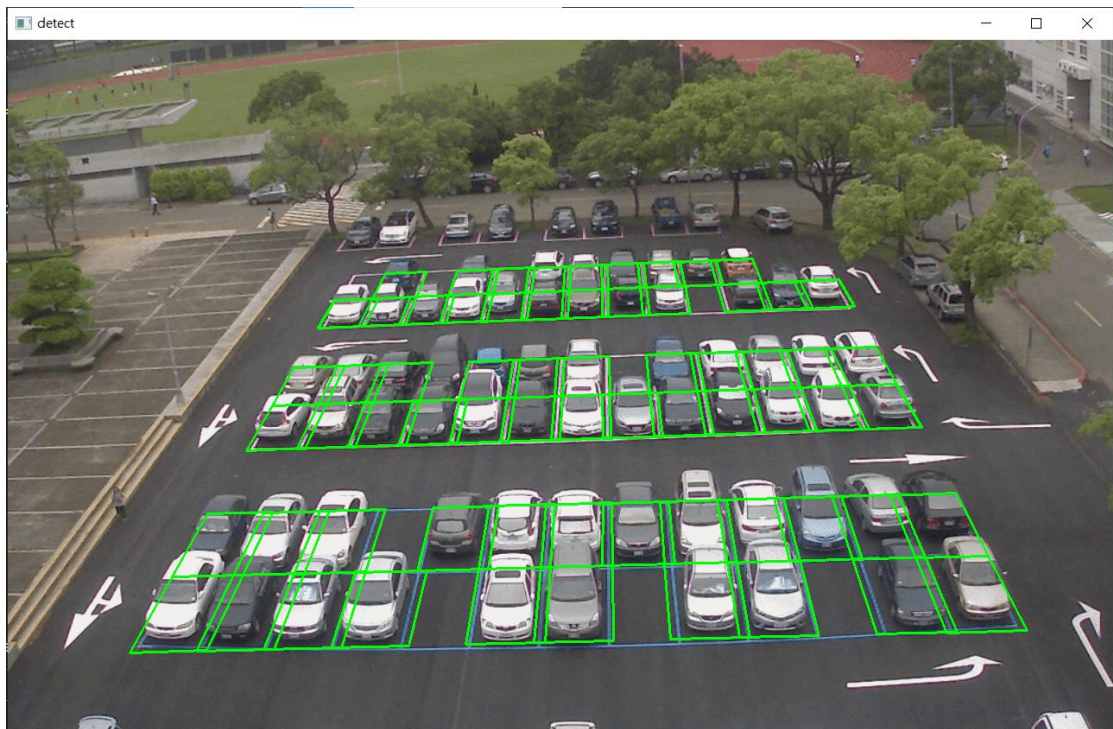
Part 3 other method

我選擇試試看 Gradient Boosting 方法來預測，程式碼整合結合在本來的 code 中。



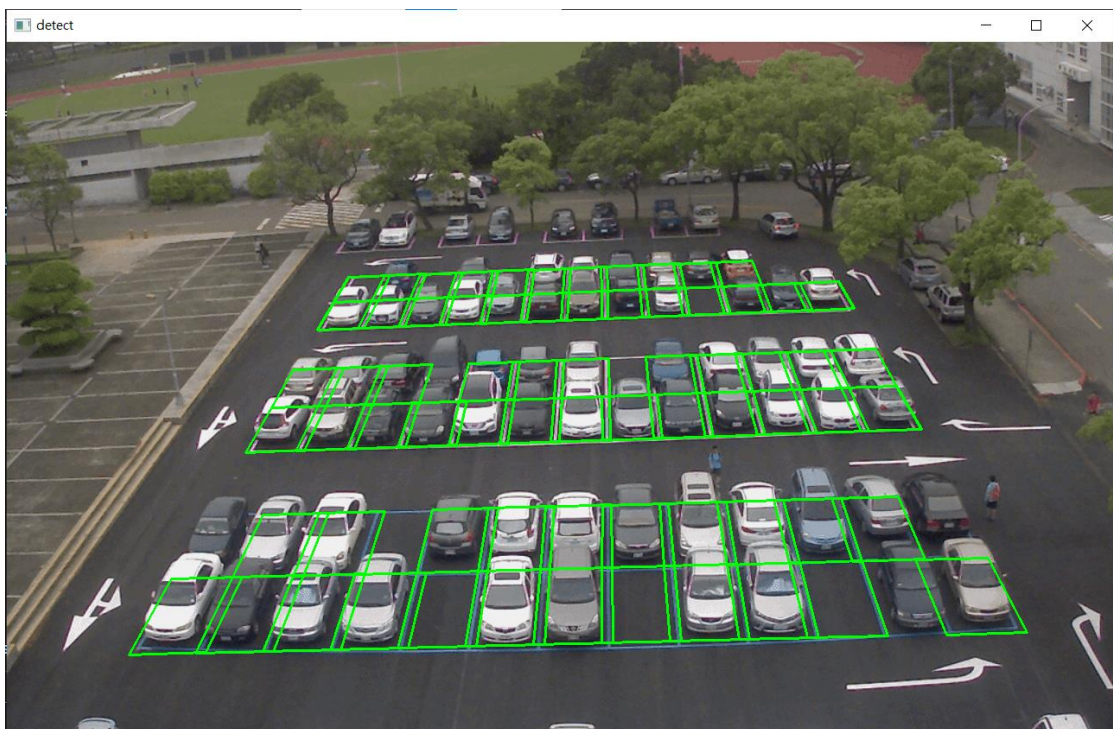
圖六： $n_estimator$ 值對各種指標的影響(GB)

與 Adaboost 相比，兩者最佳預測的狀況分別落在 GB: $n = 140$;
AB: $n = 60$ 下，GB 各項指標皆略勝一點；且 AB 在 FP 的誤判率較高，較易將沒車的车位誤判為有車。



圖七：GB 判斷圖

可以看到主要誤判出現在 FN



圖八：AB 判斷圖

FN、FP 皆有出現

遇到的問題與如何解決：這是我第一次接觸相關的知識與實作，在資料維度改變的 reshape 上吃了不少苦頭，要仔細將像素算清楚、涵式所需的參數類型仔細核對，才可以順利傳入訓練，這也仰賴於 zip、list、map 等結合運用，對我這個 python 苦手來說花費了不少時間惡補。但比起一篇一篇文章查詢與在腦中翻譯，現在有 ChatGPT 可以快速查詢與查看範例程式碼，在一無所知的前提，能夠有第一步輪廓相當實用，可以利用他提供的關鍵字尋找更多網路上的精華文章，大大降低學習所需的時間，不失為一種新型態的學習方式。